

Programmare un *computer* nel *computer* gestendo il carico cognitivo

Gabriele Pozzan
Tullio Vardanega

10 ottobre 2026, Padova



UNIVERSITÀ DI PADOVA

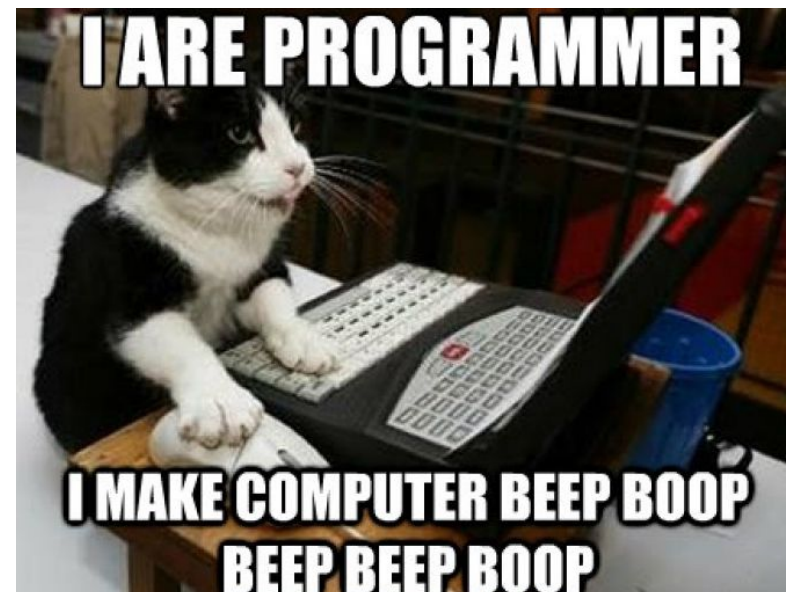
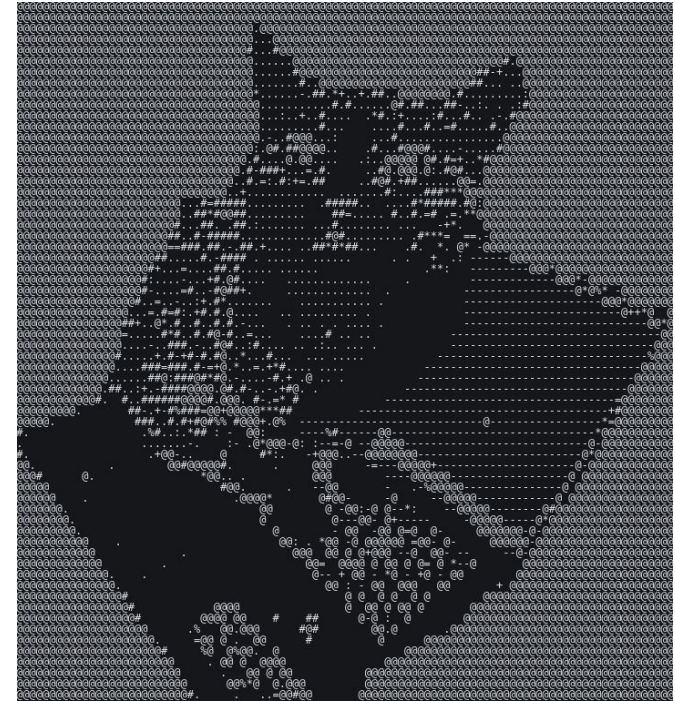
Dipartimento
di Matematica
“Tullio Levi-Civita”

[ITADINFO]

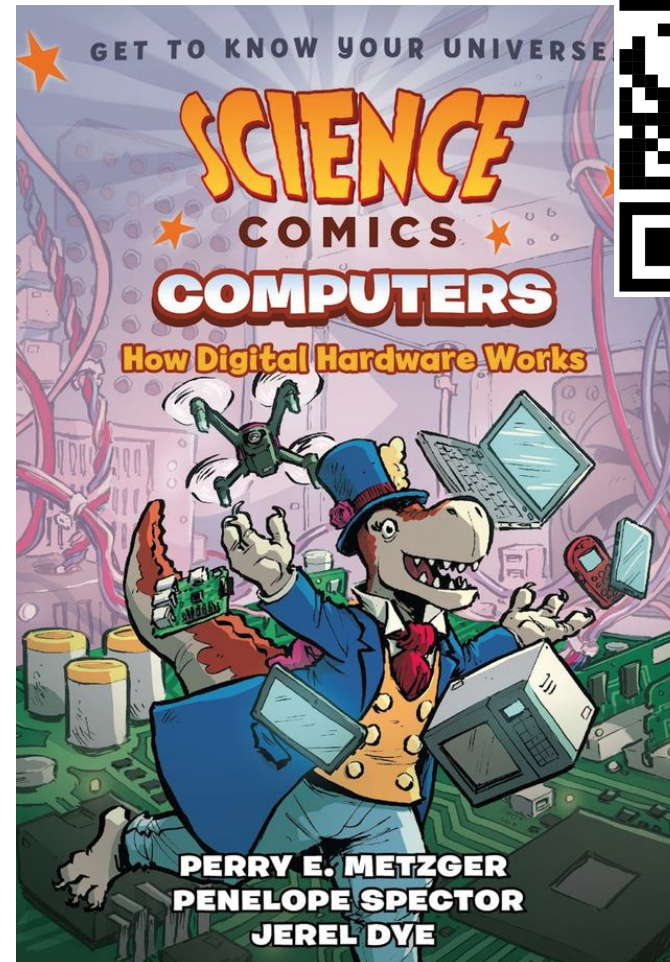
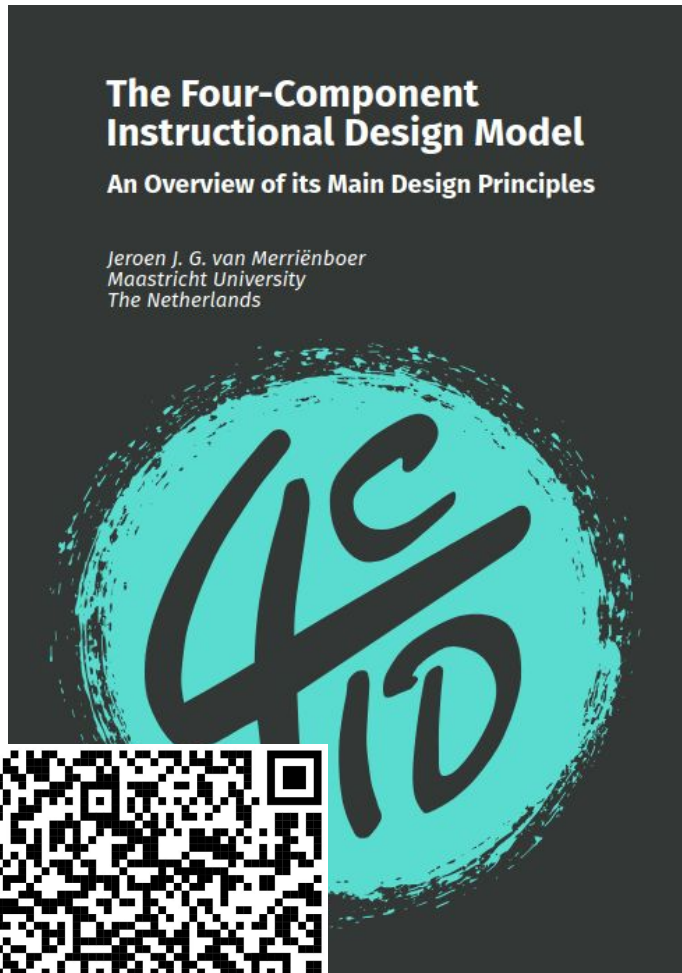
CONVEGNO ITALIANO
SULLA DIDATTICA DELL'INFORMATICA

Cosa facciamo oggi?

- In questo laboratorio **programmeremo** una **simulazione di computer** in Python
- Nel farlo vedremo alcuni concetti didattici legati alla **teoria del carico cognitivo**
- Per semplificare le cose (speriamo) useremo **Google Colab**. Per aprire il progetto avete bisogno di un **account Gmail**
 - In alternativa se volete potete scaricare direttamente il *repository* del progetto



Questo laboratorio ha due fonti principali



TQRCG



TQRCG

Premessa teorica (1)

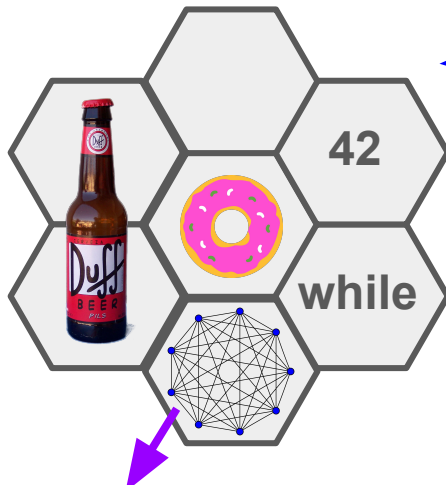
Teoria del **carico cognitivo** (John Sweller, Jeroen J. G. van Merriënboer)

Memoria di lavoro

- Limitata
 - **Spazio**: pochi elementi contemporaneamente
 - **Tempo**: mantiene gli elementi per poco tempo
- **Collo di bottiglia per l'apprendimento**

Memoria a lungo termine

- Sostanzialmente priva di limiti
- Funziona per **schemi**
- Gli schemi vengono “caricati” in memoria di lavoro



Gli **schemi** hanno basso **carico cognitivo**



Obiettivo: evitare il sovraccarico della memoria di lavoro



Cognitive Architecture and Instructional Design: 20 Years Later

John Sweller¹  • Jeroen J. G. van Merriënboer² • Fred Paas^{3,4}

Published online: 22 January 2019

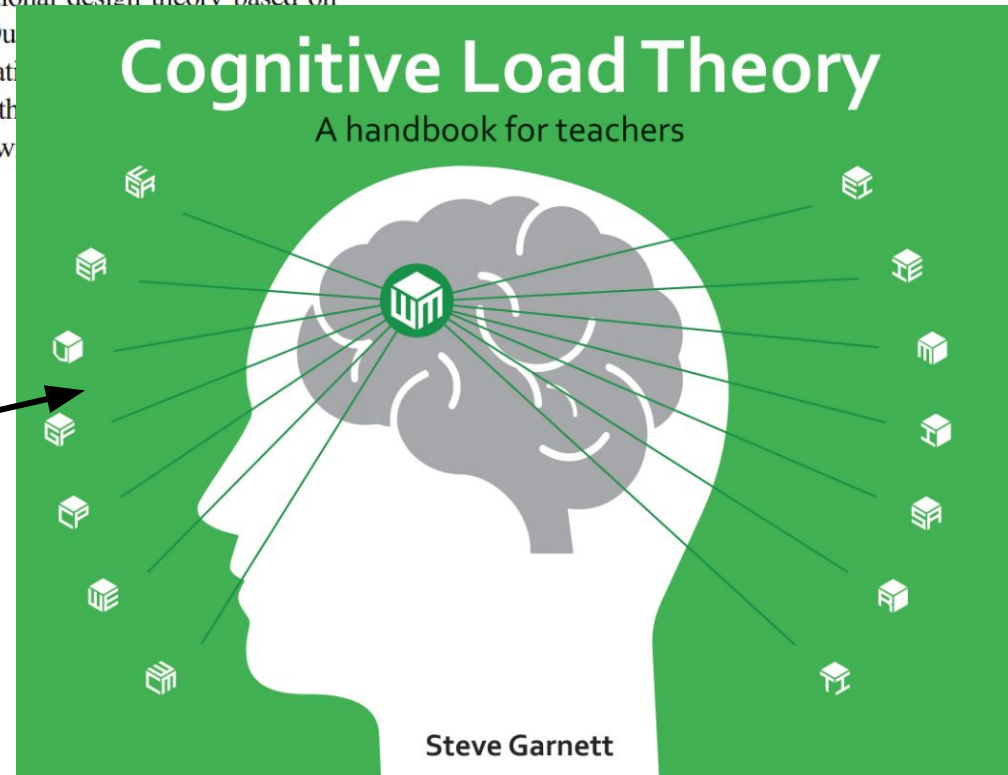
© The Author(s) 2019

Abstract

Cognitive load theory was introduced in the 1980s as an instructional design theory based on several uncontroversial aspects of human cognitive architecture. Our characteristics of working memory, long-term memory and the relationship between them are well-established for many decades prior to the introduction of the theory. The theory had had a limited impact on the field of instructional design with

Articolo accademico,
scritto in modo
comprensibile e utile
anche a livello
concreto

Manuale divulgativo,
“approvato” anche da
J. Sweller



Premessa teorica (2)



Due tipologie di **processi mentali** sono associate al pensiero che “risolve problemi”



Sistema 1 (abilità ricorrenti)

- Veloce
- Inconscio
- Automatico
- Inflessibile
- Intuitivo
- Richiede esercitazioni **frequenti** per **lunghi periodi** di tempo con l'obiettivo di **automatizzare** certe competenze di base
- Istruzioni **chiare** e applicabili facilmente
- **Riscontro** correttivo **immediato** sugli esercizi
- Esempio: esercizi di tipo “*drill*”

Sistema 2 (abilità non ricorrenti)

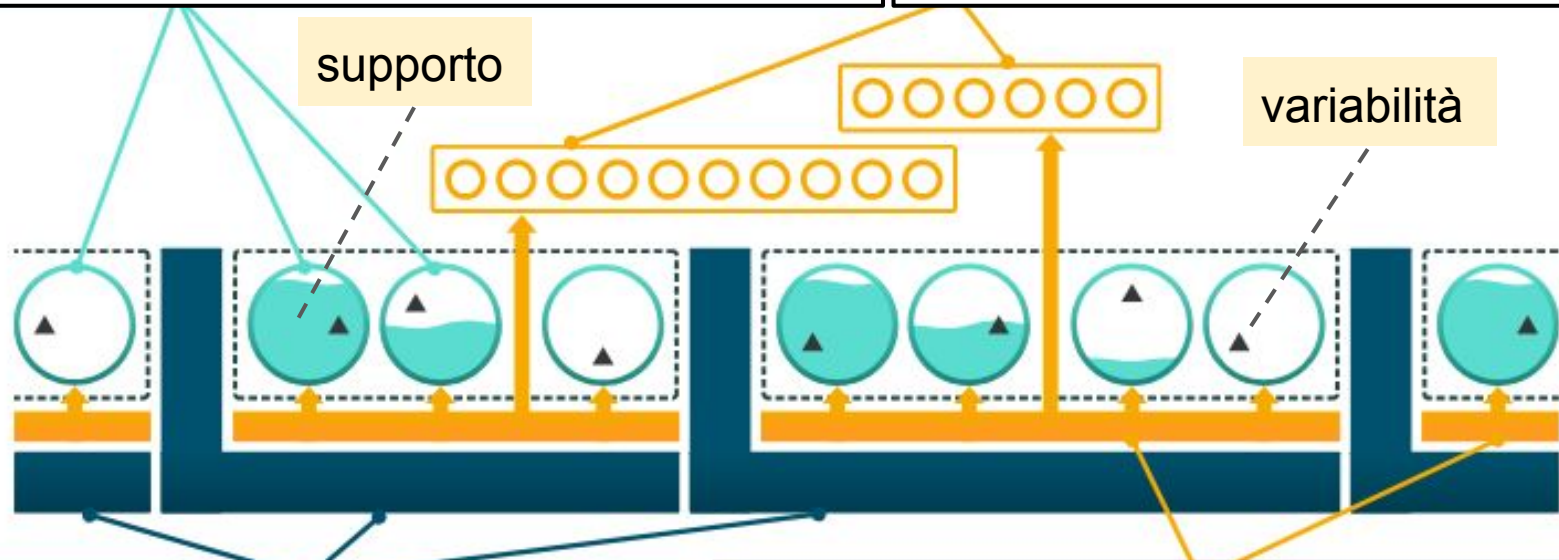
- Lento
- Conscio
- Flessibile
- “Costoso”
- Richiede **varietà** nelle esercitazioni, con l'obiettivo di **generalizzare** le abilità
- Le esercitazioni sono supportate da **informazioni di contesto**
- Enfasi sulla **riflessione** prima, durante e dopo gli esercizi (**metacognizione**)
- Esempio: esercizi di “*problem solving*”

Problemi di apprendimento (*learning task*)

- Integrano **abilità** di sistema 1 e sistema 2, **nozioni**, **attitudini**
- Modellati su problemi e situazioni **realistiche**
- **Supporto a scomparsa**, complessità crescente
- Enfasi sulla **variabilità** (apprendimento induttivo, generalizzazione)

Pratica mirata (*part-task practice*)

- Punta ad **automatizzare** alcune competenze **ricorrenti**
- Basata sulla **ripetizione**
- Ha senso dopo aver incontrato le competenze in problemi di apprendimento

**Informazioni di supporto (“parte teorica”)**

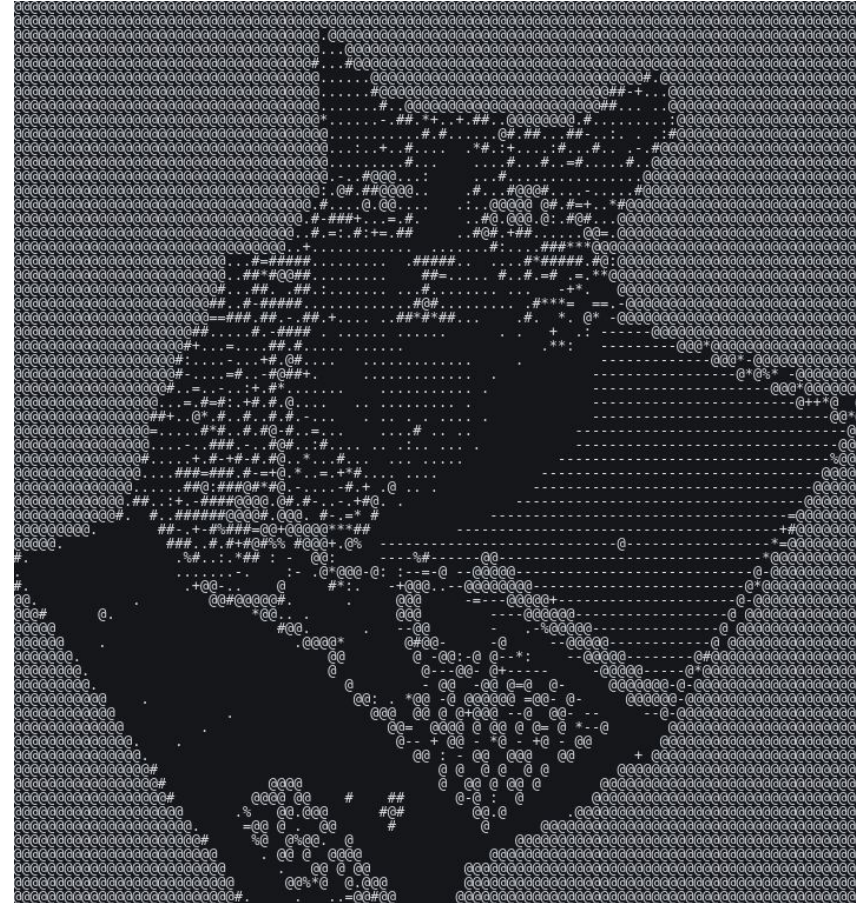
- Supportano gli aspetti **non ricorrenti** dei problemi di apprendimento
- **Strategie** (come affrontare i problemi) e **modelli** (come “capire” il dominio)
- Sempre **a disposizione**
- Specifiche per gruppi di problemi

Informazioni procedurali

- Servono per risolvere gli aspetti **ricorrenti** dei problemi
- Guida **passo-passo**, riscontro su esercizi, ecc.
- Meglio presentate **al momento in cui servono** (*just in time*, quando le altre informazioni sono ancora in memoria di lavoro)

Perché un progetto collettore?

- Contestualizza ciò che si apprende
 - **Esempio:** la prima volta che si sente parlare di una **struttura dati** non è per niente chiaro **cosa ci debba andare dentro a quella struttura dati**
- Assomiglia molto a un prodotto Informatico “completo” della “**vita vera**”
 - Il nostro obiettivo è preparare le persone alla vita vera per l'appunto
- È probabilmente più **motivante** di una serie di esercizi “usa e getta”
- In base al tipo di progetto scelto, permette di approfondire concetti di Informatica o altre discipline:
 - La trigonometria è molto importante nei videogiochi
 - Una simulazione permette di “giocare” con molti concetti di fisica
 - ...



Informazioni di supporto Architettura

Celle di memoria che hanno una **relazione stretta** con le istruzioni

INPUT

*Un **file** il cui contenuto viene copiato in **memoria** all'inizio della simulazione.*

Conserva sia **istruzioni** che **dati**

REGISTRI

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

PROCESSORE

Indirizzo istruzione

*Decodifica ed **esegue** le istruzioni
Aggiorna l'**indirizzo istruzione***

OUTPUT

*Testo che viene stampato nel **terminale** del computer che esegue la simulazione*

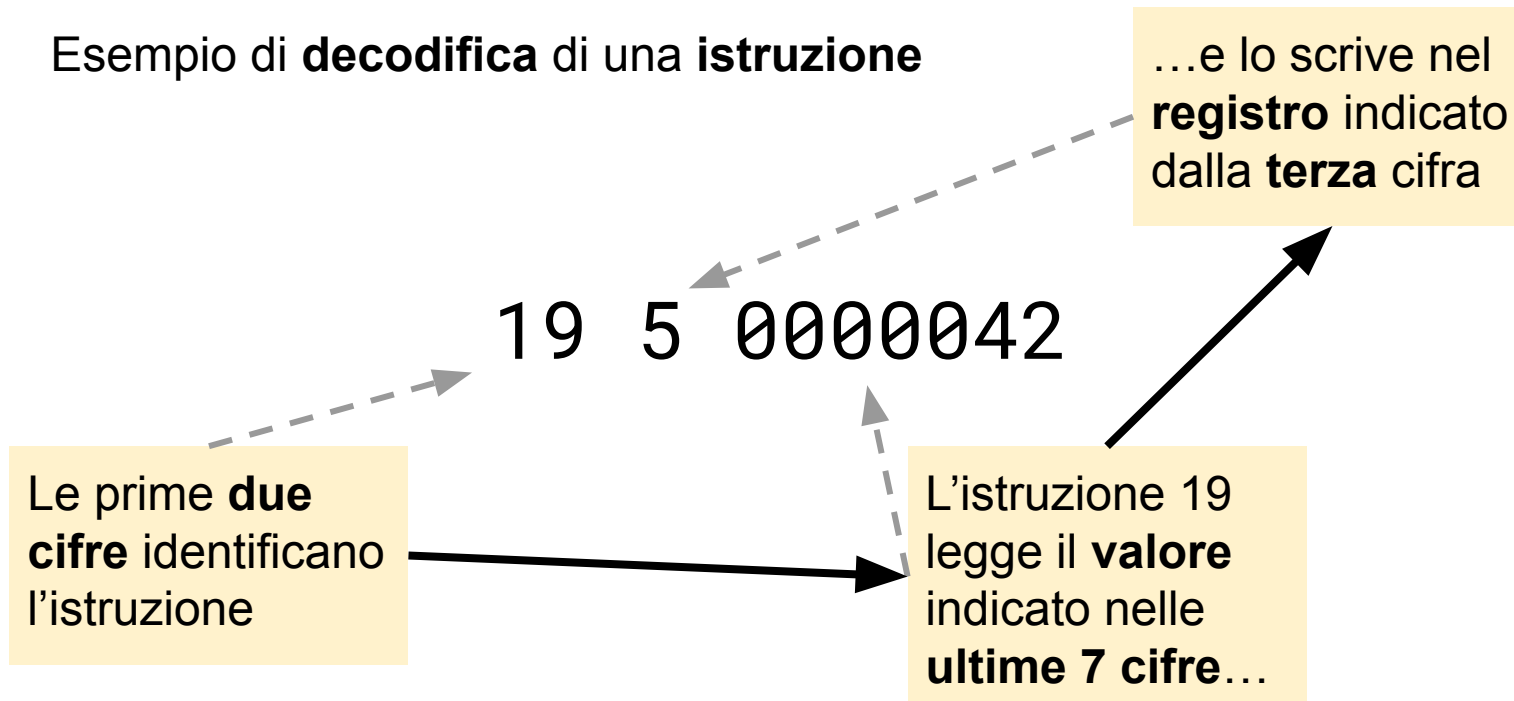
MEMORIA

0	1910000007
1	1221000000
2	4420000006
3	6120100000
4	2111100000
5	5200000001
6	0100000000
7	0000000046
8	0000000089
9	0000000070
10	0000000073
11	0000000078

Ciclo di vita del **PROCESSORE**

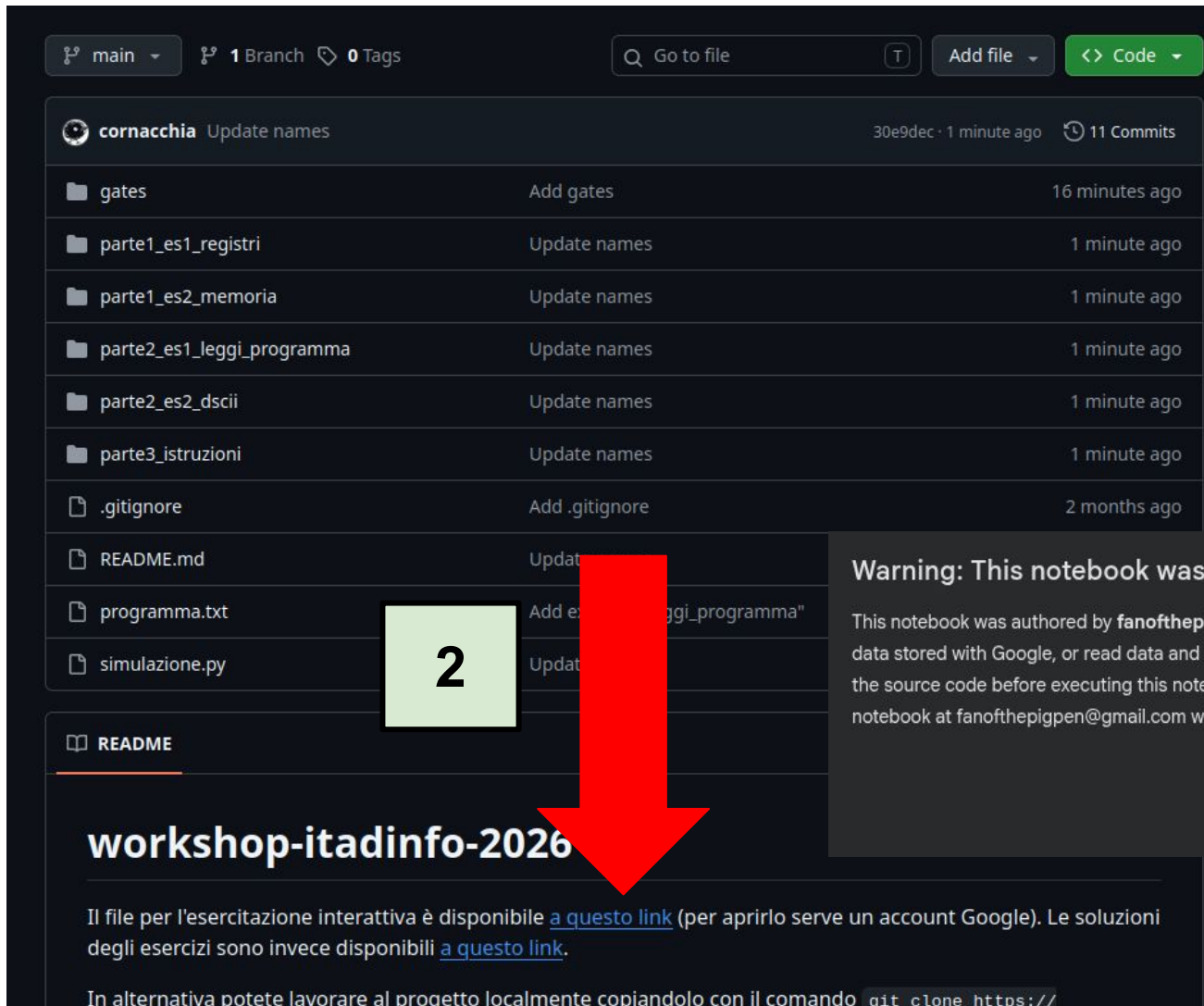
1. Legge il valore dell'**indirizzo istruzione**
2. Se il valore è uguale a -1 **termina l'esecuzione**
3. Altrimenti usa il valore letto al punto 1 per recuperare la **prossima istruzione da eseguire** dalla **memoria**
4. Aumenta di 1 l'**indirizzo istruzione** (Perchè? Perchè in questo punto?)
5. **Decodifica** ed **esegue** l'istruzione recuperata al punto 3
6. Torna al punto 1

Esempio di **decodifica** di una **istruzione**



1

<https://github.com/cornacchia/workshop-itadinfo-2026>



main 1 Branch 0 Tags

Go to file T Add file <> Code

cornacchia Update names 30e9dec · 1 minute ago 11 Commits

File/Folder	Action	Time
gates	Add gates	16 minutes ago
parte1_es1_registri	Update names	1 minute ago
parte1_es2_memoria	Update names	1 minute ago
parte2_es1_leggi_programma	Update names	1 minute ago
parte2_es2_dscii	Update names	1 minute ago
parte3_istruzioni	Update names	1 minute ago
.gitignore	Add .gitignore	2 months ago
README.md	Update	
programma.txt	Add programma.txt	
simulazione.py	Update	

2

3

Warning: This notebook was not authored by Google

This notebook was authored by fanofthepigpen@gmail.com. It may request access to your data stored with Google, or read data and credentials from other sessions. Please review the source code before executing this notebook. Please contact the creator of this notebook at fanofthepigpen@gmail.com with any additional questions.

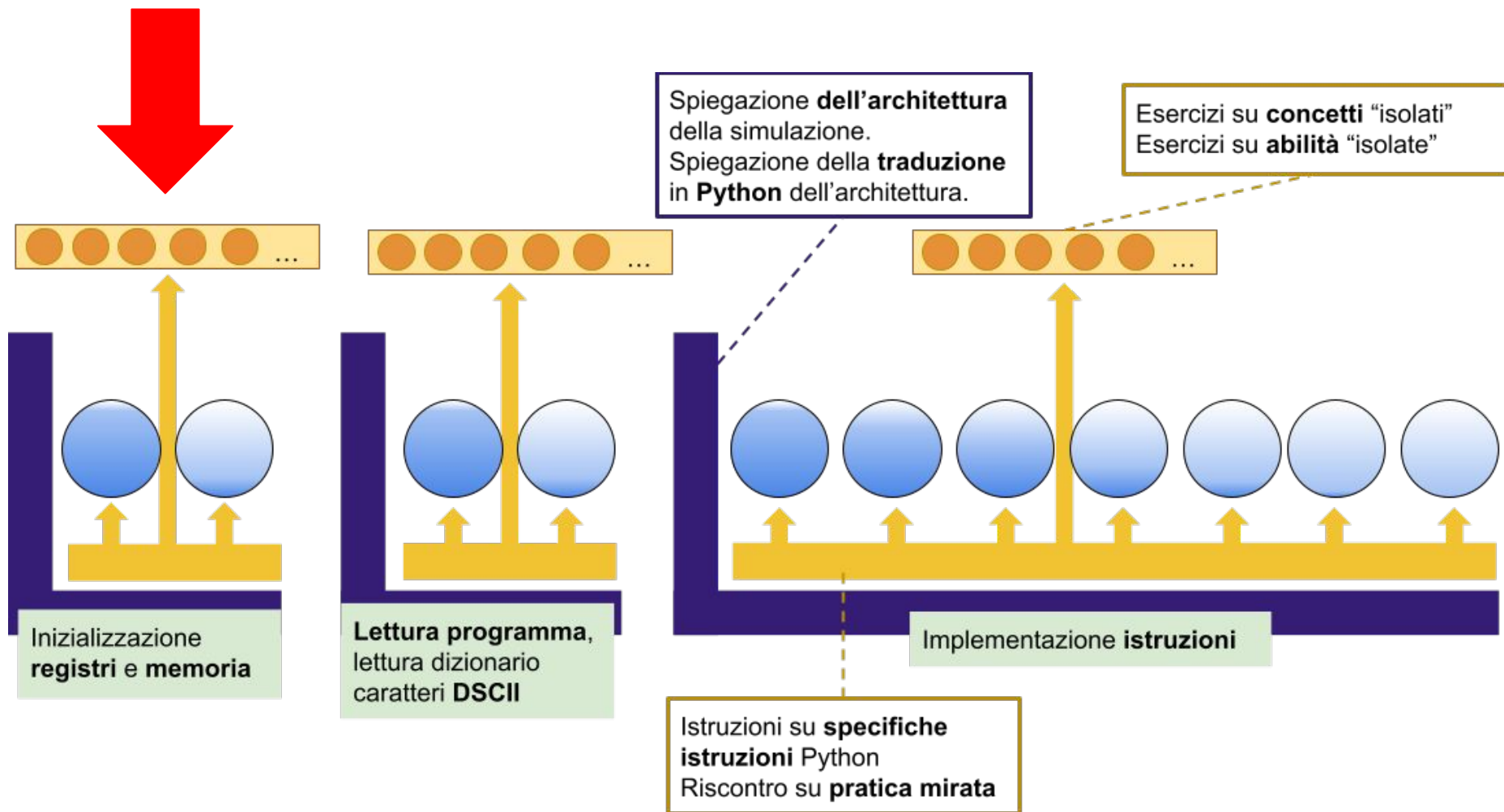
Cancel Run anyway

workshop-itadinfo-2026

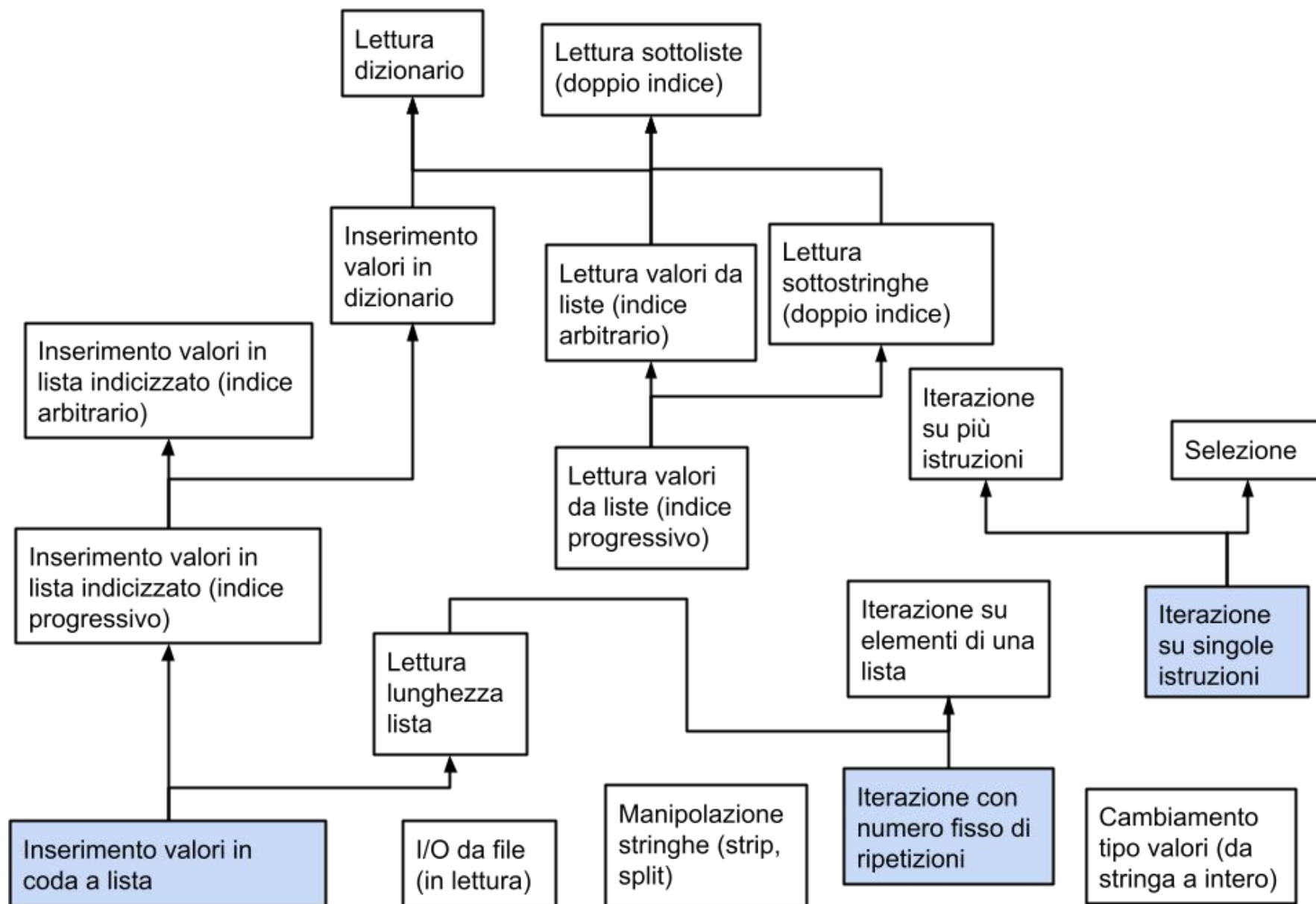
Il file per l'esercitazione interattiva è disponibile [a questo link](#) (per aprirlo serve un account Google). Le soluzioni degli esercizi sono invece disponibili [a questo link](#).

In alternativa potete lavorare al progetto localmente copiandolo con il comando `git clone https://`

Parte 1



Parte 1 - Mappa competenze



Parte 1 - Informazioni di supporto

REGISTRI	
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

MEMORIA	
0	1910000007
1	1221000000
2	4420000006
3	6120100000
4	2111100000
5	5200000001
6	0100000000
7	0000000046
8	0000000089
9	0000000070
10	0000000073
11	0000000078
12	0000000083
13	0000000075
14	0000000084

- **Registri e memoria** sono **contenitori** di più valori ai quali dobbiamo poter accedere con un **indirizzo**
- In Python li possiamo simulare con delle **liste**
- NB: questa è una scelta **arbitraria**

Esercizio 1.1

Esercizio in:

`parte1_es1_registri/registri.py`

Test: `python3 parte1_es1_registri/test.py`

Esercizio 1.1 - Registri

Formulazione del problema:

Scrivi una funzione chiamata **inizializza_registri** che restituisca una lista di **10 elementi**. Gli elementi devono essere valori **interi** ed essere inizializzati tutti con il valore **0**.

Esempio di soluzione in pseudocodice:

funzione **inizializza_registri**:

 Crea una lista vuota chiamata **registri**

 Ripeti per **10** volte:

 Aggiungi il valore **0** alla lista

 Restituisci la lista **registri**

NB: questa soluzione forse non è la più efficiente o la più idiomatica

Good news: sappiamo già come fare esercizi successivi!

Obiettivi:

1. Studiare lo **schema algoritmico** della soluzione (serve per l'esercizio successivo). Separiamo inizialmente algoritmo e codice eseguibile.
2. Tradurre in linguaggio eseguibile Python lo pseudocodice. Focalizziamo l'attenzione sulla **sintassi**.

Informazioni procedurali

Se la soluzione ha errori vedrete:

```
gabriele@omen-gabriele:~/progetti/workshop-itadinfo-2026$ python3 registri/test.py
TypeError: object of type 'NoneType' has no len()

=====
ERROR: test_valori (__main__.TestRegistri.test_valori)
-----
Traceback (most recent call last):
  File "/home/gabriele/progetti/workshop-itadinfo-2026/registri/test.py", line 16, in test_valori
    self.assertTrue(len(reg) > 0, 'La lista restituita non contiene elementi.')
    ~~~~~^
TypeError: object of type 'NoneType' has no len()

=====
FAIL: test_restituisce_lista (__main__.TestRegistri.test_restituisce_lista)
-----
Traceback (most recent call last):
  File "/home/gabriele/progetti/workshop-itadinfo-2026/registri/test.py", line 8, in test_restituisce_lista
    self.assertIsInstance(reg, list, 'La funzione inizializza_registri non restituisce una lista')
AssertionError: None is not an instance of <class 'list'> : La funzione inizializza_registri non restituisce una lista

-----
Ran 3 tests in 0.001s

FAILED (failures=1, errors=2)
```

Quello che volete vedere:

```
gabriele@omen-gabriele:~/progetti/workshop-itadinfo-2026$ python3 registri/test.py
...
Ran 3 tests in 0.000s
OK
```

NB: Questo è un esempio di sviluppo **guidato da test** (*test driven*)

Formulazione del problema:

Scrivi una funzione chiamata **inizializza_memoria** che accetti un **parametro intero** chiamato **dimensione** e restituisca una lista con un numero di elementi uguale a **dimensione**. Gli elementi devono essere valori **interi** ed essere inizializzati tutti con il valore **0**.

Obiettivi:

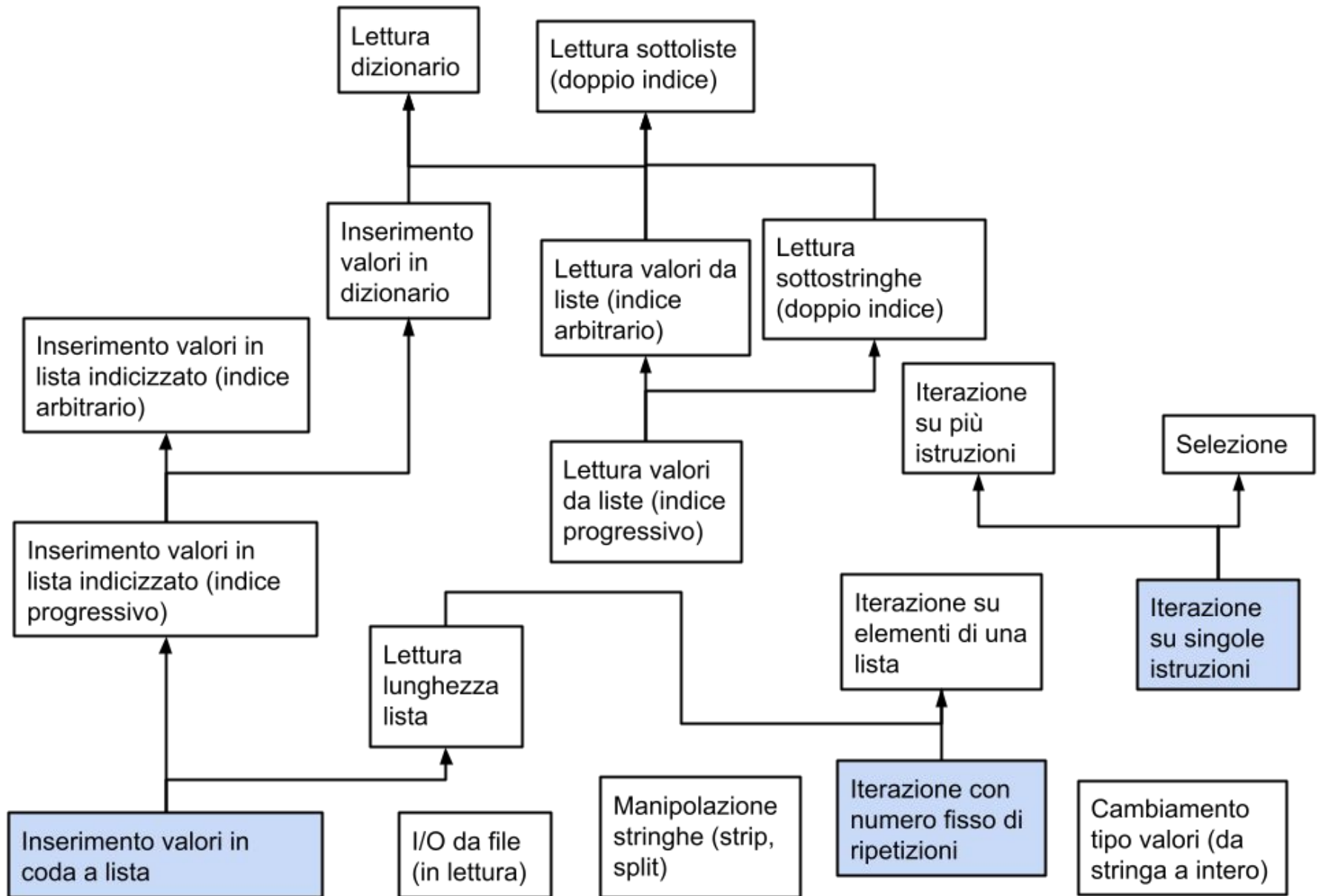
1. Adattare ed applicare lo **schema algoritmico** dell'esercizio precedente. Cosa cambia rispetto all'inizializzazione dei registri? (separiamo soluzione algoritmica da implementazione)
2. Implementare la soluzione in linguaggio eseguibile Python.

- Identifichiamo quali sono le competenze che ci aspettiamo possano essere **automatizzate**.
 - Costruiamo esercizi piccoli, con *feedback* immediato, che possano essere fatti frequentemente (es: 10 domande a risposta multipla a inizio lezione per riscaldamento?)
 - Il 4C/ID suggerisce di puntare a questi obiettivi, in questo ordine: **1) accuratezza, 2) velocità, 3) simultaneità con altre operazioni**
-
- Questo laboratorio è dedicato soprattutto ai problemi di apprendimento.
 - Piuttosto di provare un insieme limitato e arbitrario di esercizi di pratica mirata proviamo a pensare a un metodo per “generarli”.

Ci servono due pezzi:

1. Una **scomposizione** delle **sotto-competenze** necessarie per compiere l'atto macroscopico (tutto quello che mi serve per “programmare”)
2. Una scomposizione di come ci **interfacciamo** mentalmente a queste sotto-competenze (il codice lo leggi, lo ricordi, lo progetti, lo scrivi, ...)

Pratica mirata - Pezzo 1



Tassonomia di Bloom (*)

- È un elenco di **obiettivi educativi** che cercano di rappresentare livelli di comprensione progressivamente più **profondi**
- Può essere usata per **stimare** a che punto sia arrivata la comprensione di un determinato concetto
- Nel nostro caso può essere usata per **suggerire possibili esercizi**

I livelli sono:

1. **Ricordare** (riconoscere) - recuperare informazioni dalla memoria a lungo termine (“sapere a memoria”)
2. **Capire** (interpretare, classificare, spiegare) - comprendere il significato delle informazioni
3. **Applicare** (eseguire, implementare) - applicare una procedura
4. **Analizzare** (decomporre) - capire le relazioni tra informazioni di un dominio
5. **Valutare** (criticare, monitorare, testare) - giudicare criticamente a partire da informazioni
6. **Creare** (pianificare, generare, produrre) - produrre un pensiero/prodotto originale

(*) Thompson, E. et al. (2008). Bloom's Taxonomy for CS Assessment. *Proceedings of the tenth ACE2008, ACM: Wollongong*.
<https://dl.acm.org/doi/10.5555/1379249.1379265>

	Ricordare	Capire	Applicare	Analizzare	Valutare	Creare
Iterazione con numero fisso di ripetizioni	Riconoscere un ciclo scritto correttamente (sintassi)	Tradurre un ciclo da pseudocodice a codice eseguibile	Fare tracing di un programma con cicli	Trovare codice equivalente	Riconoscere problemi che possono essere risolti con cicli	Risolvere problemi con cicli
Inserimento valori in coda a lista	Identificare il metodo corretto da una lista	Identificare con la posizione di un elemento inserito in una lista (all'interno di una sequenza)	...	...	...	...
...	...	...	...	...	...	...

Iterazione con numero fisso di ripetizioni

Ricordare

1. Quali di queste parole chiave sono normalmente usate per la sintassi di un ciclo? `def`, `range`, `list`, `for`, `while`, `return`, `global`, `import`, `in`, `assert`

Capire

1. Quale variabile viene modificata ad ogni ripetizione del ciclo?
2. Quali di questi cicli ripetono sicuramente un numero fisso di volte?
 - a. `for i in range(5)`
 - b. `for i in range(a)`
 - c. `for i in [1, 2, 3]`
 - d. `for i in mia_lista`

```
j = 2
for i in range(5):
    print(j)
```

Versioni successive possono diventare sempre più precise e dettagliate.
Conviene **esplicitare la macchina nozionale** di riferimento a monte.

Applicare

```
j = 2
for i in range(5):
    print(j)
```

1. Riempi la tabella di tracing

	Passo 1	Passo 2	Passo 3	Passo 4	Passo 5	...
i	x	x	0	..	..	..
j	2	2	2	..	..	..

Iterazione con numero fisso di ripetizioni

Analizzare

- Quali dei seguenti programmi sono equivalenti a questo?

```
j = 2
for i in range(5):
    print(j)
```

```
j = 2
i = 0
print(j)
i = 1
print(j)
i = 2
print(j)
i = 3
print(j)
i = 4
print(j)
i = 5
print(j)
```

```
j = 2
print(j)
i = 0
print(j)
i = 1
print(j)
i = 2
print(j)
i = 3
print(j)
i = 4
print(j)
i = 5
```

```
j = 2
for i in range(2):
    print(j)

for i in range(2, 5):
    print(j)
```

Valutare

- Quali dei seguenti problemi possono essere risolti con un unico ciclo con un numero fisso di ripetizioni?
 - Stampa la tabellina del 3
 - Stampa tutte le tabelline da 1 a 10
 - Conta quante volte appare la lettera "a" in una stringa di testo
 - Disegna un quadrato usando i caratteri #

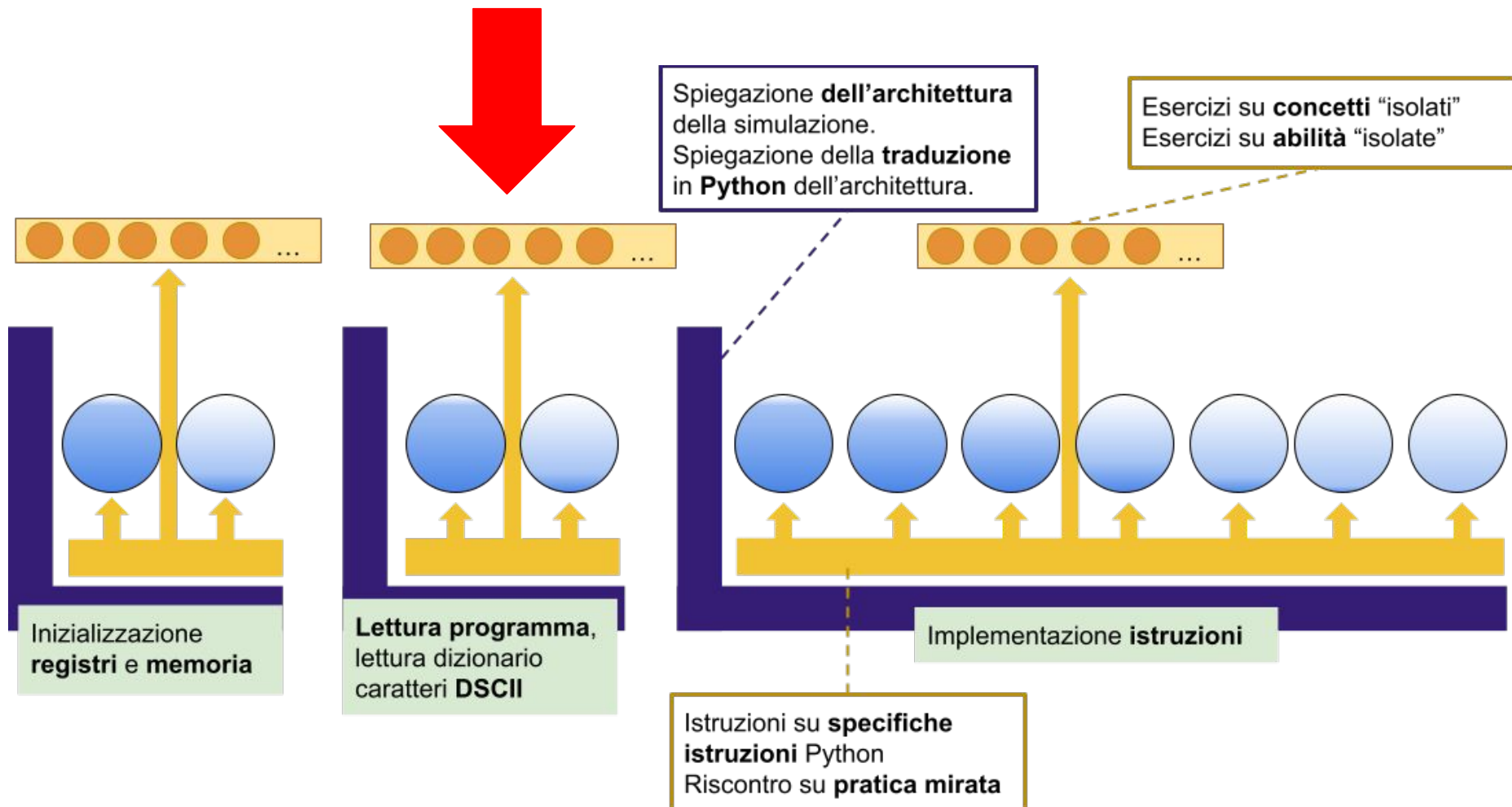
```
#####
#      #
#####
```

Creare

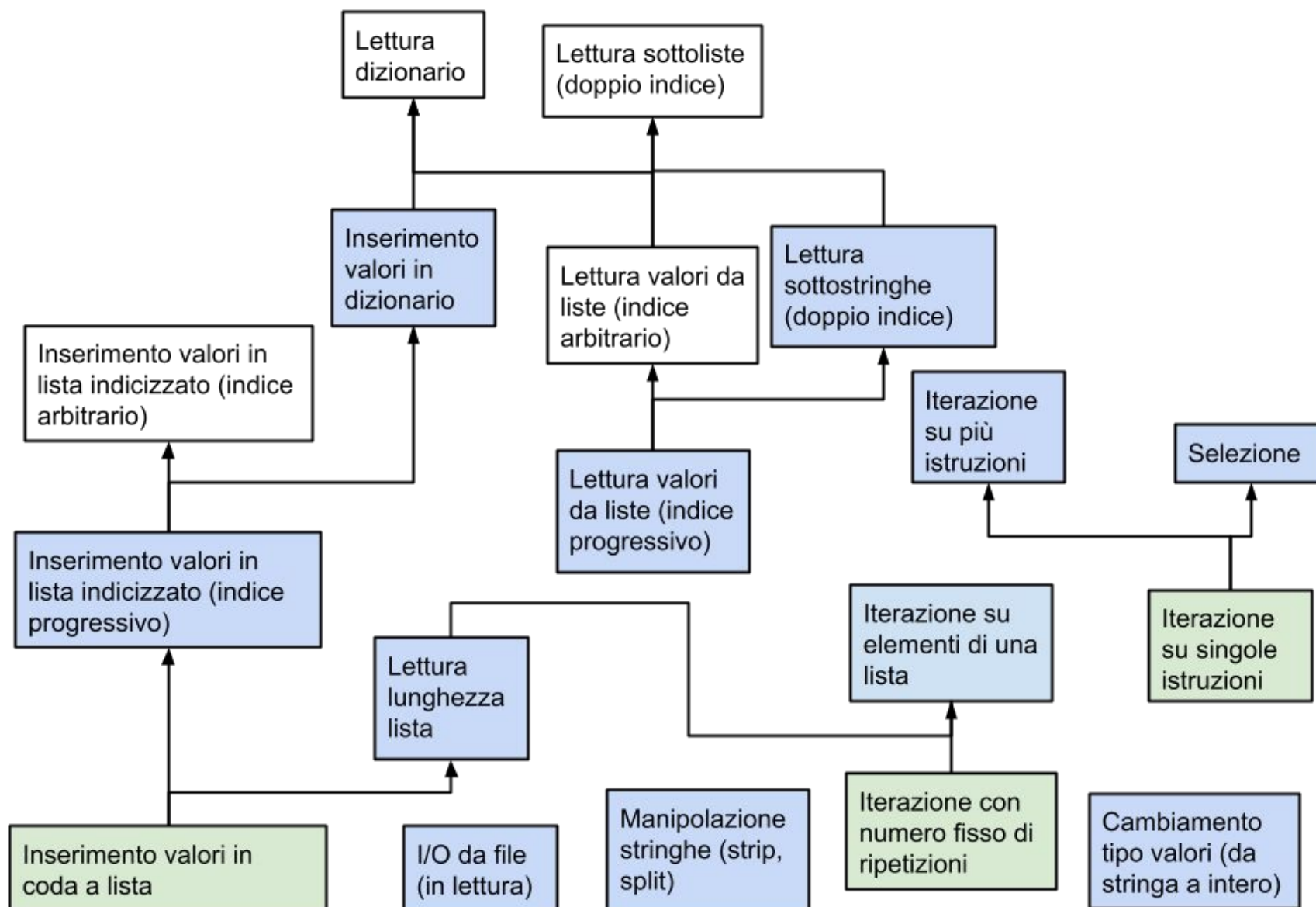
```
mia_lista = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k']  
for i in range(      ):  
    print(mia_lista[i])
```

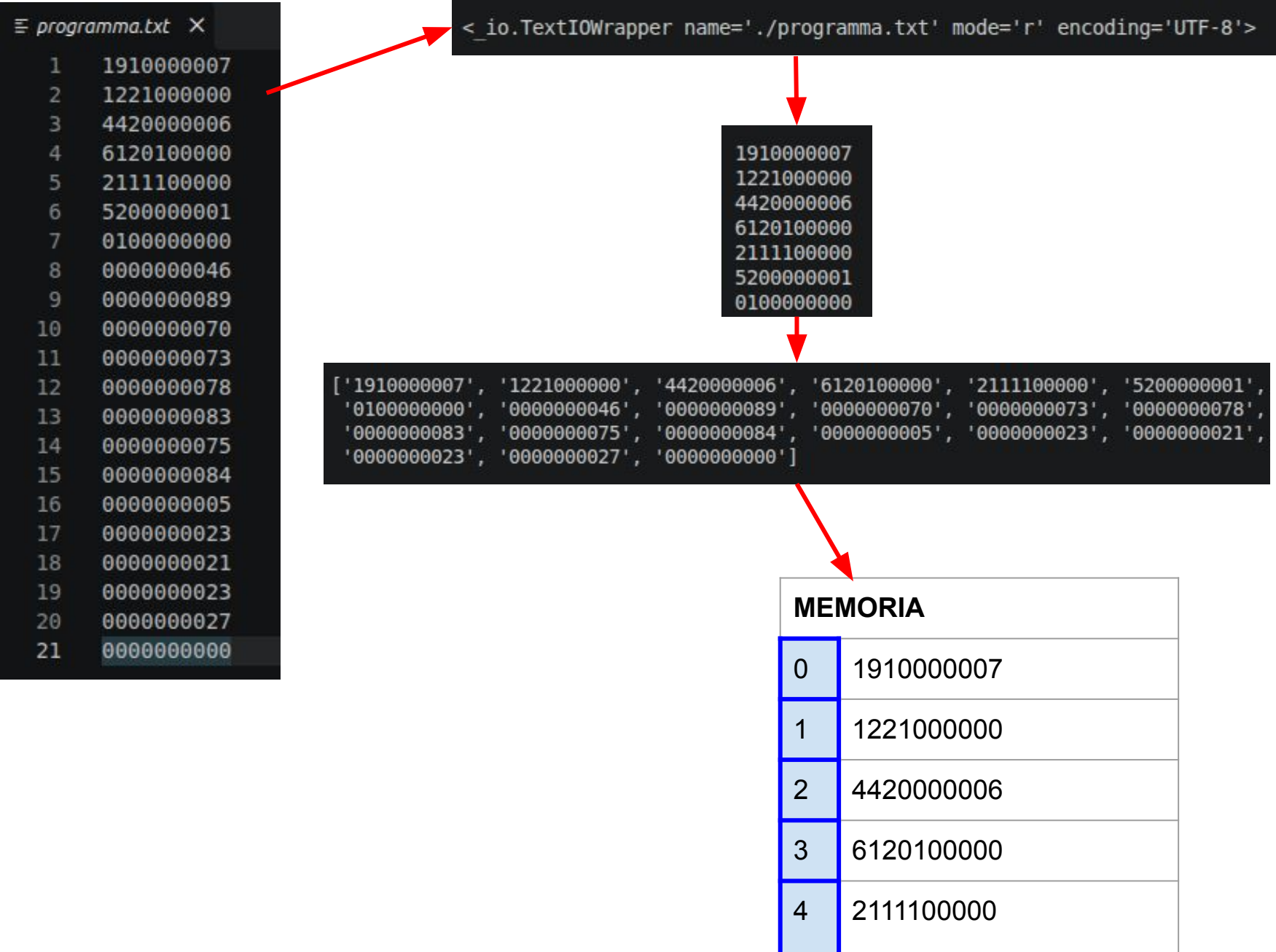
1. Cosa devi scrivere dentro alle parentesi di range per ottenere i seguenti risultati?
 - a. a, b, c, d, e, f, g, h, i, j, k
 - b. k, j, i, h, g, f, e, d, c, b, a
 - c. a, b, c
 - d. a, c, e, g, i, k
 - e.
 - f. a
 - g. f
 - h. f, e
2. Implementa il programma senza usare un ciclo (e viceversa)

Parte 2



Parte 2 - Mappa competenze





Esercizio 2.1

Esercizio in:

parte2_es1_leggi_programma/leggi_programma.py

Test: python3 parte2_es1_leggi_programma/test.py

Formulazione del problema:

Scrivi una funzione chiamata **leggi_programma_in_memoria** che accetti come parametri il **nome di un file** e il riferimento alla **memoria** della simulazione.

Possiamo assumere che il file conterrà un programma in linguaggio macchina, ogni riga del file corrisponde a una singola istruzione.

La funzione deve leggere le istruzioni e copiarle in ordine nelle celle della memoria della simulazione, una istruzione per cella, a partire dalla cella numero 0.

Attenzione: ricordiamo che la memoria deve contenere **numeri**

PRIMM

Predict
Run
Investigate
Modify
Make

```
21 def leggi_programma_in_memoria (nome_file, memoria):
22     with open(nome_file, 'r') as file_programma:
23         contenuto = file_programma.read()
24         lista_contenuto = contenuto.split('\n')
25         for i in range(5):
26             print(lista_contenuto[i])
27     return memoria
```

Esercizio 2.1

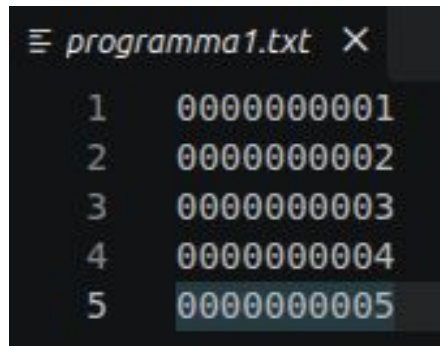
PRIMM: PREDICT / RUN (1)

Predict - Leggi il programma e prevedi quale sarà l'esito della sua esecuzione

Run - Esegui il programma e verifica la correttezza della tua previsione

```
def leggi_programma_in_memoria (nome_file, memoria):
    with open(nome_file, 'r') as file_programma:
        contenuto = file_programma.read()
        lista_contenuto = contenuto.split('\n')
        for i in range(5):
            print(lista_contenuto[i])
    return memoria

# Le righe successive servono per le fasi predict/run
# Commentarle alla fine dell'esercizio
memoria = [0 for i in range(100)]
leggi_programma_in_memoria('./parte2_es1_leggi_programma/materiale_test/programma2.txt', memoria)
```



	programma1.txt
1	0000000001
2	0000000002
3	0000000003
4	0000000004
5	0000000005

Esercizio 2.1

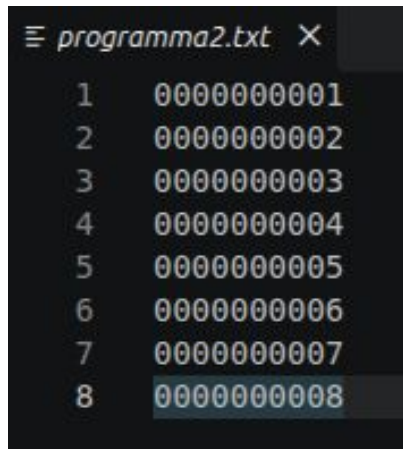
PRIMM: PREDICT / RUN (2)

Predict - Leggi il programma e prevedi quale sarà l'esito della sua esecuzione

Run - Esegui il programma e verifica la correttezza della tua previsione

```
def leggi_programma_in_memoria (nome_file, memoria):
    with open(nome_file, 'r') as file_programma:
        contenuto = file_programma.read()
        lista_contenuto = contenuto.split('\n')
        for i in range(5):
            print(lista_contenuto[i])
    return memoria

# Le righe successive servono per le fasi predict/run
# Commentarle alla fine dell'esercizio
memoria = [0 for i in range(100)]
leggi_programma_in_memoria('./parte2_es1_leggi_programma/materiale_test/programma2.txt', memoria)
```



```

≡ programma2.txt ×
1  0000000001
2  0000000002
3  0000000003
4  0000000004
5  0000000005
6  0000000006
7  0000000007
8  0000000008

```

Esercizio 2.1

PRIMM: INVESTIGATE

Analizza il programma, approfondisci alcuni suoi aspetti, identifica eventuali errori/mancanze/miglioramenti possibili

```
21 def leggi_programma_in_memoria (nome_file, memoria):
22     with open(nome_file, 'r') as file_programma:
23         contenuto = file_programma.read()
24         lista_contenuto = contenuto.split('\n')
25         for i in range(5):
26             print(lista_contenuto[i])
27     return memoria
28
29 # Le righe successive servono per le fasi predict/run
30 # Commentarle alla fine dell'esercizio
31 leggi_programma_in_memoria('./leggi_programma/materiale_test/programma2.txt')
```

Pratica mirata

1. Cosa vuol dire 'r' alla riga 22?
2. In che porzione (righe) del programma posso interagire con la variabile **file_programma**?
3. Cosa contiene la variabile **contenuto** dopo l'esecuzione della riga 23?
4. Cosa contiene la variabile **lista_contenuto** dopo l'esecuzione della riga 24?
5. Cosa manca per completare l'esercizio?
 - a. Cosa succede se il programma eccede la capacità della memoria?

Esercizio 2.1

PRIMM: MODIFY

Modifica il programma per raggiungere uno o più obiettivi stabiliti nelle fasi precedenti (o esplicitamente indicati dall'esercizio).

Esercizio in: `parte2_es1_leggi_programma/leggi_programma.py`
Test: `python3 parte2_es1_leggi_programma/test.py`

Formulazione del problema:

Scrivi una funzione chiamata **leggi_programma_in_memoria** che accetti come parametri il **nome di un file** e il riferimento alla **memoria** della simulazione.

Possiamo assumere che il file conterrà un programma in linguaggio macchina, ogni riga del file corrisponde a una singola istruzione.

La funzione deve leggere le istruzioni e copiarle in ordine nelle celle della memoria della simulazione, una istruzione per cella, a partire dalla cella numero 0.

Parte 2 - Informazioni di supporto

- Il fumetto da cui è tratta l'architettura della simulazione descrive una codifica numeri - caratteri alfabetici chiama **DSCII**
- Noi la leggeremo da un file e la conserveremo in un **dizionario Python** (anche questa è una scelta arbitraria)

Code	Character	Code	Character	Code	Character	Code	Character	Code	Character
0	<i>NUL</i>	1	<i>[reserved]</i>	2	<i>[reserved]</i>	3	<i>[reserved]</i>	4	<i>[newline]</i>
5	<i>[space]</i>	6	!	7	"	8	#	9	\$
10	%	11	&	12	'	13	(	14	)
15	*	16	+	17	,	18	-	19	.
20	/	21	0	22	1	23	2	24	3
25	4	26	5	27	6	28	7	29	8
30	9	31	:	32	;	33	<	34	=
35	>	36	?	37	@	38	A	39	B
40	C	41	D	42	E	43	F	44	G
45	H	46	I	47	J	48	K	49	L
50	M	51	N	52	O	53	P	54	Q
55	R	56	S	57	T	58	U	59	V

Esercizio 2.2

PRIMM: MAKE

Crea un nuovo programma usando i concetti appresi durante le fasi PRIMM precedenti

Esercizio in: `parte2_es2_dscii/dscii.py`

Test: `python3 parte2_es2_dscii/test.py`

Formulazione del problema:

Scrivi una funzione chiamata **inizializza_dscii** che restituisca un **dizionario** nel quale ogni chiave sia un **codice DSCII** e ogni valore il **carattere DSCII** corrispondente.

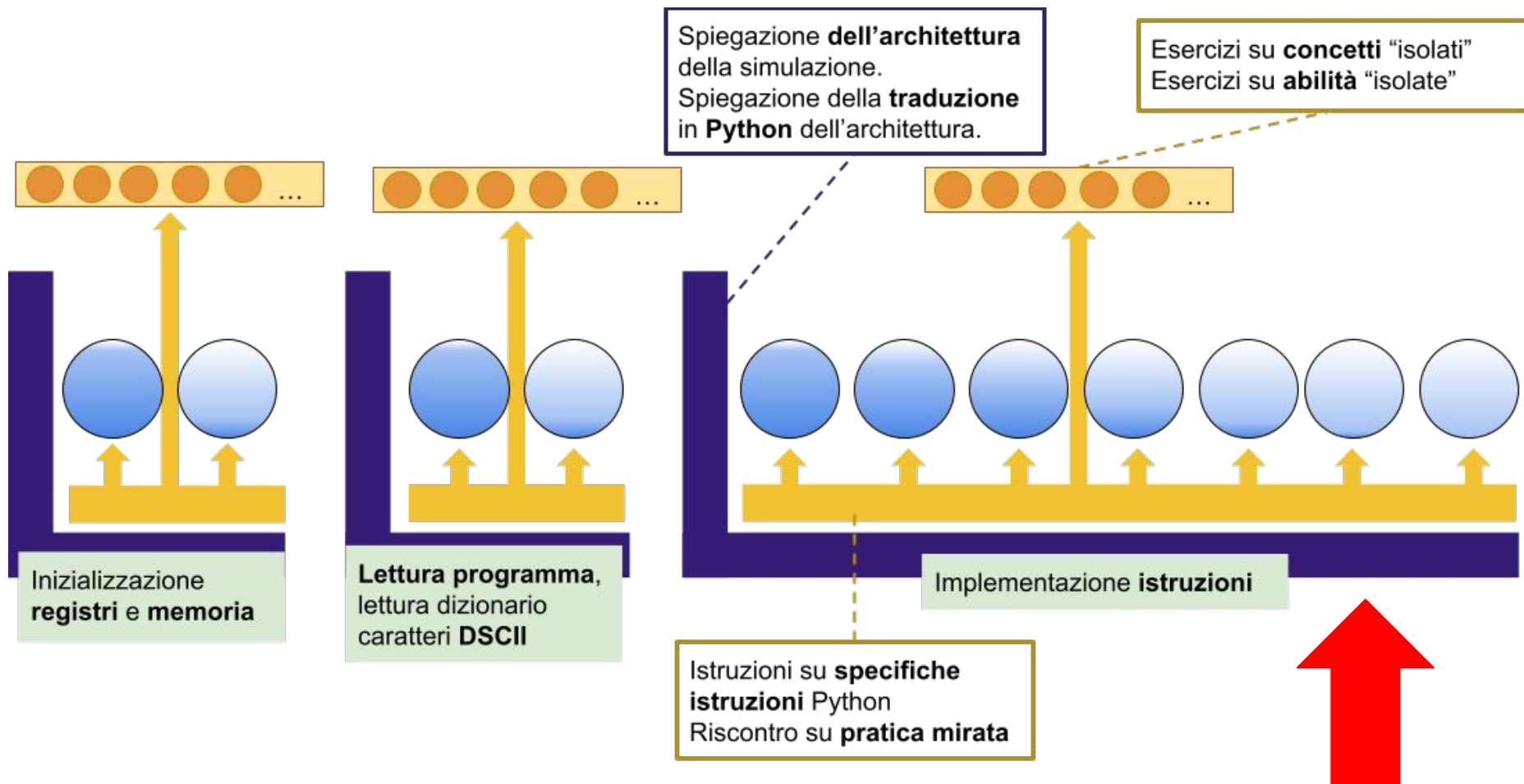
I codici e caratteri DSCII sono scritti nel file **dscii/dscii.txt** dove ogni riga contiene un **codice numerico** (primi due caratteri della riga) e un **simbolo di carattere** (resto della riga).

I caratteri vanno salvati **senza spazi** intorno, ad esempio così "a" e non così " a ".

Il carattere **[spazio]** va sostituito con uno spazio " ".

Puzzle di Parson

La soluzione viene fornita in "ordine sparso" e l'obiettivo è rimetterla in ordine

**Parte 3**

[ITADINFO]
CONVEGNO ITALIANO
SULLA DIDATTICA DELL'INFORMATICA



Parte 3 - Informazioni di supporto

```
def esegui_12 (indirizzo_istruzione, registri, memoria, istruzione_da_eseguire):  
    # codice qui  
    return indirizzo_istruzione, registri, memoria, (-1, -1)
```

output

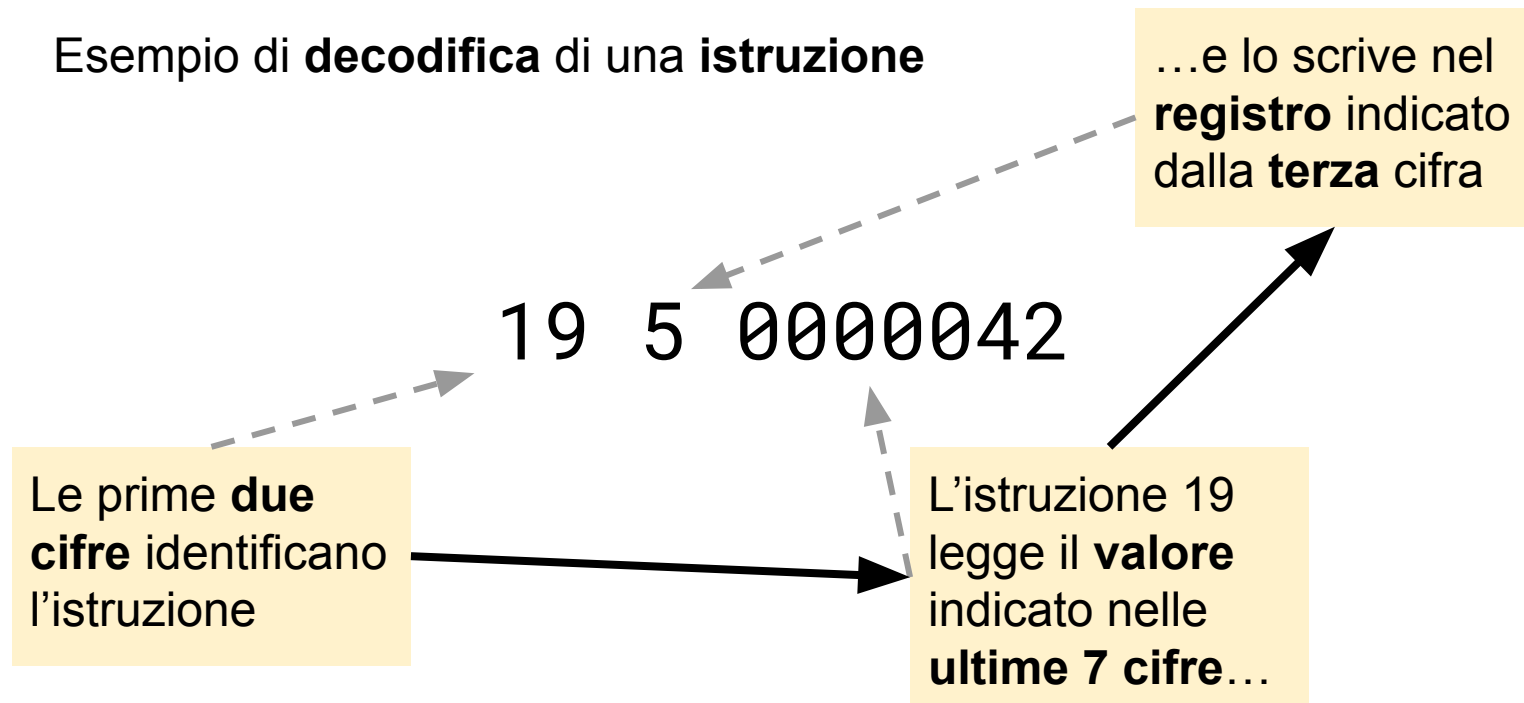
REGISTRI	
0	34
1	0
2	12
3	13
4	67
5	34
6	11
7	7890
8	6
9	2

Istruzione corrente

Indirizzo prossima
istruzione: 5

MEMORIA	
...	
4	1263700000
...	
20	42
21	4
22	34
23	43
24	34
25	56
...	

Esempio di **decodifica** di una **istruzione**



Esercizio 3.1

Istruzione 12

Somma il **contenuto del registro** indicato nella **quarta** cifra dell'istruzione al **valore** contenuto nella **quinta** cifra e usa il risultato come **indirizzo**.

Copia il contenuto della **memoria** all'**indirizzo** calcolato nei passaggi precedenti nel **registro** indicato dalla **terza** cifra dell'istruzione.

REGISTRI	
0	34
1	0
2	12
3	13
4	67
5	34
6	11
7	7890
8	6
9	2

Istruzione corrente

Indirizzo prossima
istruzione: 5

Pratica mirata

Dato questo stato di partenza e questa istruzione sopra, qual è l'esito che ci aspettiamo?

MEMORIA	
...	
4	1263700000
...	
20	42
21	4
22	34
23	43
24	34
25	56
...	

Esercizio 3.2

Istruzione 19

Copia il **valore** indicato nelle cifre dalla **quarta** alla **decima** dell'istruzione (estremi compresi) nel **registro** indicato dalla **terza** cifra dell'istruzione.

REGISTRI	
0	34
1	0
2	12
3	13
4	67
5	34
6	11
7	7890
8	6
9	2

Istruzione corrente

Indirizzo prossima
istruzione: 5

Pratica mirata

Dato questo stato di partenza e questa istruzione sopra, qual è l'esito che ci aspettiamo?

MEMORIA	
...	
4	1920042000
...	
20	42
21	4
22	34
23	43
24	34
25	56
...	

Esercizio 3.3

Istruzione 21

Somma il **contenuto** del **registro** indicato dalla **quarta** cifra dell'istruzione al **valore** contenuto nella **quinta** cifra dell'istruzione.

Copia il risultato nel **registro** indicato dalla **terza** cifra dell'istruzione.

REGISTRI	
0	34
1	0
2	12
3	13
4	6
5	34
6	11
7	7890
8	6
9	2

Istruzione corrente

Indirizzo prossima
istruzione: 5

Pratica mirata

Dato questo stato di partenza e questa istruzione sopra, qual è l'esito che ci aspettiamo?

MEMORIA	
...	
4	2174300000
...	
20	42
21	4
22	34
23	43
24	34
25	56
...	

Esercizio 3.4

Istruzione 01

Stop. Interrompe la simulazione scrivendo il valore -1 sull'**indirizzo** (prossima) **istruzione**.

REGISTRI	
0	34
1	0
2	12
3	13
4	67
5	34
6	11
7	7890
8	6
9	2

Istruzione corrente

Indirizzo prossima
istruzione: 5

Pratica mirata
Dato questo stato di
partenza e questa
istruzione sopra, qual è
l'esito che ci aspettiamo?

MEMORIA	
...	
4	01000000000
...	
20	42
21	4
22	34
23	43
24	34
25	56
...	

Esercizio 3.5

Istruzione 44

Controlla se il **contenuto** del **registro** indicato dalla **terza** cifra dell'istruzione è zero. Se è zero, copia il **valore** contenuto nelle cifre dalla **quarta** alla **decima** dell'istruzione (estremi compresi) nell'**indirizzo (prossima) istruzione**.

Questa è una istruzione di "salto" condizionale che permette, ad esempio, di implementare cicli.

REGISTRI	
0	34
1	0
2	12
3	13
4	67
5	34
6	11
7	7890
8	6
9	2

Istruzione corrente

Indirizzo prossima
istruzione: 5

Pratica mirata
Dato questo stato di partenza e questa istruzione sopra, qual è l'esito che ci aspettiamo?

MEMORIA	
...	
4	441000000042
...	
20	42
21	4
22	34
23	43
24	34
25	56
...	

Esercizio 3.6

Istruzione 52

Copia il **valore** indicato nelle cifre dalla **quarta** alla **decima** dell'istruzione (estremi compresi) nell'**indirizzo (prossima) istruzione**.

Questa è una istruzione di "salto" non condizionale, che può essere usata ad esempio per eseguire procedure.

REGISTRI	
0	34
1	0
2	12
3	13
4	67
5	34
6	11
7	7890
8	6
9	2

Istruzione corrente

Indirizzo prossima
istruzione: 5

Pratica mirata

Dato questo stato di partenza e questa istruzione sopra, qual è l'esito che ci aspettiamo?

MEMORIA	
...	
4	521000000042
...	
20	42
21	4
22	34
23	43
24	34
25	56
...	

Esercizio 3.7

Istruzione 52

Il **valore** contenuto nelle cifre **quarta** e **quinta** dell'istruzione indica un canale di comunicazione (ad esempio: terminale, file, stampante, ecc.). Il contenuto del **registro** indicato nella **terza** cifra dell'istruzione è inviato a quel canale.

REGISTRI	
0	34
1	0
2	42
3	13
4	67
5	34
6	11
7	7890
8	6
9	2

Istruzione corrente

Indirizzo prossima
istruzione: 5

Pratica mirata

Dato questo stato di partenza e questa istruzione sopra, qual è l'esito che ci aspettiamo?

(???, ???)

MEMORIA	
...	
4	6120100000
...	
20	42
21	4
22	34
23	43
24	34
25	56
...	

0	1910000007
1	1221000000
2	4420000006
3	6120100000
4	2111100000
5	5200000001
6	0100000000
7	0000000046
8	0000000089
9	0000000070
10	0000000073
11	0000000078
12	0000000083
13	0000000075
14	0000000084
15	0000000005
16	0000000023
17	0000000021
18	0000000023
19	0000000027
20	0000000000

0 - Scrivi 7 nel registro 1

1 - Somma il contenuto del registro 1 a 0 e usa il risultato come indirizzo (in questo caso la riga 7 del programma), poi copia il valore della memoria a quell'indirizzo nel registro 2

2 - Se il registro 2 contiene 0, salta alla istruzione 6

3 - Manda il contenuto del registro 2 al canale di comunicazione 1 (terminale)

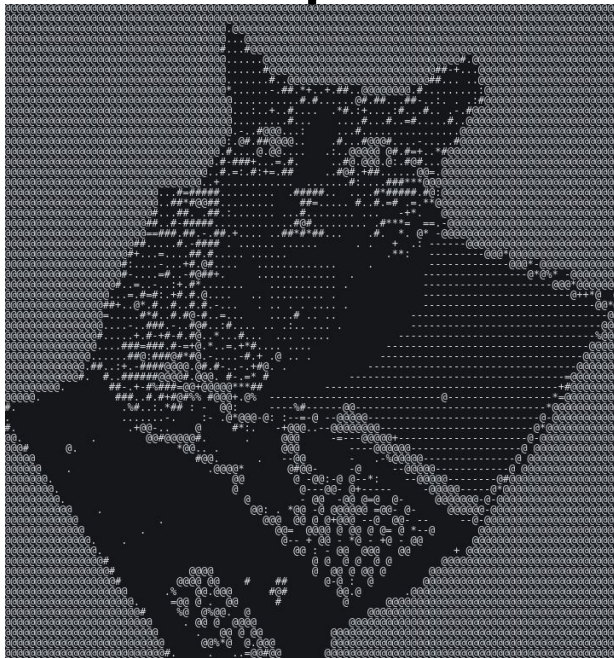
4 - Somma il contenuto del registro 1 (7) a 1 e copia il risultato nel registro 1

5 - Salta sempre indietro all'istruzione 1

6 - Termina il programma

Come espandere questa simulazione

Attività con programmi scritti in linguaggio macchina ed eseguiti dalla simulazione.



Abbiamo implementato un sottoinsieme delle possibili istruzioni.
Manca un meccanismo di memoria “duratura” (disco fisso).
...

Implementazione più a “basso livello” di **componenti e operazioni fondamentali**.
Con **AND**, **NOT**, **NAND** e un **FLIP-FLOP** si può simulare la memoria!